# Druid

## 一个实时分析数据仓库

Druid-China

作者：

**Fangjin Yang**
Metamarkets Group, Inc.
fangjin@metamarkets.com

**Xavier Léauté**
Metamarkets Group, Inc.
xavier@metamarkets.com

**Gian Merlino**
Metamarkets Group, Inc.
gian@metamarkets.com

**Eric Tschetter**
echeddar@gmail.com

**Nelson Ray**
ncray86@gmail.com

**Deep Ganguli**
Metamarkets Group, Inc.
*deep@metamarkets.com*

译者：



**个人简介：**
致力于实时/离线大数据分析！实时大数据分析系统 **Druid** 在中国应用落地拓荒者。
热心开源事业，Zookeeper 管理系统 Zookeeper-Edit、Zookeeper 集群一键安装脚本 Zookeeper-Cluster Installer 开发者。
曾就职于 YeahMobi(Druid 官方列出的 20 个将 Druid 用于商业生产的组织机构中唯一一个中国公司),从事在线广告业务数据分析，负责 **DSP**(Demand-Side Platform)部分核心系统研发、参与**原生广告** Android SDK 研发工作。
现就职于携程旅行网大数据部门。
**联系方式：**zhoushineyoung@hotmail.com

Druid-China

# 版权声明

本文原著作者 Fangjin Yang、Eric Tschetter、Xavier Léauté、Nelson Ray、Gian Merlino、Deep Ganguli，原著版权归其原作者所有。译文版权归译者所有。

允许个人或其他组织以学习为目的的硬拷贝和使用。暂不开放任何以商业为目的的分发和制作出售。

以其他方式/目的进行的复制、再发布，需预先取得授权许可。

转载须注明出处。



Druid-China

# 译者序

Druid 是一个为大型(设计为 PB 级别)数据集上实时探索查询而设计的开源数据分析和存储系统，提供极具成本效益并且永远在线(24x7x365x10)的实时数据摄入和任意数据处理，并且在面对代码部署、机器故障以及其他意外情况时能保证系统集群正常运行。

Druid 自发布以来，Github 上代码的活跃度一直很高，已经有很多组织和个人参与到该项目的开发中来。因此 Druid 在过去的一年多时间里一直保持着很高的版本发布频次。截至本文译本完成时，Druid 已经释放出 0.8.0 的稳定版本，该版本在集群的稳定性和查询性能上已经有了相当的提升，同时也提供了“简单的”界面支持。

本文是 Druid 官方提供的论文(WhitePaper)的中文译版，主要介绍 Druid 的核心概念和架构设计原理。

由于目前 Druid 相关的文字资料相当稀少，除了其官方提供的一些指导文档，本文也是最珍贵的文字资料之一。

如果你想深入了解 Druid 的运行原理，或者是你正在打算、或者是已经将 Druid 应用于商业生产环境，作为调优的参考文档，希望本文能够给您提供一定的帮助。

作为中国大陆首批将 Druid 用于生产的团队成员，深知其过程之坎坷，资源之稀缺。如果你想就相关或类似问题展开深入探讨，欢迎来信交流。

虽然已经尽力保证翻译的准确性，但囿于个人水平有限，如有翻译不妥之处，欢迎批评指正。

专用交流群：255250113(Druid-China)， 欢迎加入探讨。

2015 年 9 月 1 日 于上海

Druid-China

# 目录


摘要 ..... 1

分类和主题描述 ..... 1

关键字 ..... 1

1.引言 ..... 1

2.问题定义 ..... 2

3.架构 ..... 3

3.1 Real-Time 节点 ..... 4

3.1.1 可用性和可扩展性 ..... 5

3.2 Historical 节点 ..... 6

3.2.1 层 ..... 7

3.2.2 可用性 ..... 8

3.3 Broker 节点 ..... 8

3.3.1 缓存 ..... 8

3.3.2 可用性 ..... 8

3.4 Coordinator 节点 ..... 9

3.4.1 规则 ..... 9

3.4.2 负载均衡 ..... 9

3.4.3 复制 ..... 10

3.4.4 可用性 ..... 10

4.存储格式 ..... 10

4.1 数据过滤索引 ..... 11

4.2 存储引擎 ..... 12

5.查询 API ..... 13

6.性能 ..... 14

6.1 生产中的查询性能 ..... 15

6.2 基于 TPC-H 数据的基准查询测试 ..... 17

6.3 数据摄入(Ingestion)性能 ..... 20


Druid-China

7.生产环境中的 Druid。 ..... 21
查询模式 ..... 21
多租户 ..... 21
节点失效 ..... 21
数据中心宕机 ..... 22
7.1 运行监控 ..... 22
7.2 将 Druid 和流式处理程序相结合 ..... 22
7.3 多数据中心分布 ..... 22
8.相关工作 ..... 23
9.总结 ..... 23
10.致谢 ..... 23
11.参考文献 ..... 24

Druid-China

# 摘要

Druid 是一个设计用于对大数据集的实时探查分析(Real-Time exploratory analytics)的开源数据仓库。该系统融合了列式存储布局，分布式、无共享的体系架构，和先进的次秒延迟(sub-second latencies)随机探查十亿行表的索引结构。在本书中，我们将描述 Druid 的体系结构，并详细介绍 Druid 如何支持快速聚合，灵活过滤，和低延迟数据摄入。

# 分类和主题描述

H.2.4 [数据库管理]: 系统—分布式数据库

# 关键字

分布式; 实时; 容错; 高可用; 开源; 分析; 列式存储; 联机分析处理 (OLAP)

# 1.引言

近年来，互联网技术的发展创造了一个机器生成的事件的激增浪潮。个别的，这些事件包含最小化的低值信息。由于从大量事件中提取语意(meaning)需要时间和资源，很多公司倒宁愿选择忽略这些数据。尽管已经有了处理基于事件的数据(Event-based data)的基础架构(如 IBM 的 Netezza[37], HP(惠普)的 Vertica[5], 还有 EMC(易安信)的 Green plum[29])，但是它们大部分售价高昂，并且只定位于能买得起的那些公司。几年前，谷歌推出了利用商业硬件来索引网页和分析日志的 Map-Reduce [11] 机制。不久之后，Hadoop [36] 项目产生，它主要根据原始 Map-Reduce 论文的见地设计实现。目前，Hadoop 被许多组织机构或公司部署用于大规模日志数据的存储和分析。Hadoop 在帮助一些公司做低值(low-value)事件流到高值(high-value)聚合结果转化的一系列应用(比如 商业智能 BI 和 A-B 测试)上卓有成效。

和许多优秀的系统一样，Hadoop 已经打开了我们重新认识问题的新世界。

具体来说，Hadoop 擅长存储并提供大规模数据的访问，但是在数据访问速度上，它不提供任何性能保证。 此外，虽然 Hadoop 是一个高可用系统，但是在高并发负载压力下性能会下降。 最后， 虽然 Hadoop 适合存储数据，但是它并不是数据摄入，并使摄入的数据立即可读的最佳选择。

在 Metamarkets 产品开发的早期，我们遇到了所有这些问题并且意识到 Hadoop 是一个很优秀的后台、批处理和数据仓库系统。然而，作为一个公司，要有在高并发环境(1000+用户)下，产品级查询性能和数据可用性方面的保证。这种情况下，Hadoop 并不能够满足我们的需求。在那期间，我们调研了不同的解决方案，在尝试了关系型数据库管理系统和 NoSQL 体系架构之后，我们发现了一个结论——在开源的世界里，对于我们的情形，没有任何一个能够完全依赖的系统。最后，我们创建了 Druid，一个开源的，分布式的，面向列式存储的，可实时分析的数据仓库。Druid 和 OLAP 系统[30, 35, 22] 在很多方面具有相似性：交互式查询系统[28]，主内存数据库[14]，和众所周知的分布式数据仓库[7, 12, 23]。分布式和查询模型也借用了目前这一代的搜索基础架构[25, 3, 4]。

本文将介绍 Druid 的体系架构，探索在创建一个能提供托管服务的，不间断的，生产系统的过程当中所做的各种设计决策；和在帮助告知所有那些面临类似问题的人一个可能的解决办法上所做的尝试。

Druid 已经在几个技术公司 [2] 的生产环境中部署。

本文的结构如下：

第 2 部分，我们首先做问题描述；第 3 部分，从数据流动的角度详细阐述系统架构；第 4 部分，讨论如何及为什么数据被转换成二进制的形式；第 5 部分，简要描述查询 API；第 6 部分，目前的性能测试结果；第 7 部分，以在生产环境中部署/运行 Druid 来结束课程学习；第 8 部分，相关工作。

# 2.问题定义

Druid 最初被设计用于解决数据摄入和探查大批量事务性事件(log data)。这种形式的时间序列数据通常存在于联机分析处理 (OLAP)工作流中，并且，数据的特性是附加重很大。例如，加入数据如表 1 所示。表 1 包含了维基百科(Wikipedia)中已经存在的将要被编辑的数据。每当一个用户编辑维基百科中的一个页面，一个包含和本次编辑行为相关的元数据(Metadata)的事件将被产生。这个元数据(Metadata)由 3 个不同的组件组成。首先，有一个时间戳列用来标识编辑行为何时被执行；其次，有一组维度列，标识编辑行为的各种属性，如 被编辑的页面，执行编辑动作的用户，和该用户的地理位置；最后，是一组可被聚合的，含值(通常是数值类型)的指标列，如 一次编辑行为中被增加或被删除的字符数。

| Timestamp | Page | User name | Gender | City | Characters Added | Characters Removed |
|---|---|---|---|---|---|---|
| 2011-01-01T01:00:00Z | Justin Bieber | Boxer | Male | San Francisco | 1800 | 25 |
| 2011-01-01T01:00:00Z | Justin Bieber | Reach | Male | Waterloo | 2912 | 42 |
| 2011-01-01T02:00:00Z | Ke$ha | Helz | Male | Calgary | 1953 | 17 |
| 2011-01-01T02:00:00Z | Ke$ha | Xeno | Male | Taiyuan | 3194 | 170 |

**表 1：Druid 示例数据，维基百科(Wikipedia)上出现过的编辑**

我们的目标是通过这些数据迅速地计算深入分析和聚合。我们想要回答诸如 “页面 Justin Bieber 被来自旧金山(San Francisco)的男性编辑过多少次？”和“在一个月的时间跨度里，来自卡尔加里的人们增加的字符的平均数是多少？”类似的问题。我们也想通过任意维度组合查询，并以次秒(Sub-second)级延迟返回查询。

现存的开源关系型数据库管理系统(RDBMS)和 NoSQL 类 key/value 数据仓库不能为交互式应用程序[40] 提供一个低延迟的数据摄入和数据查询平台，而正是由于存在现实状况和

需求之间的矛盾，促进了 Druid 的发展。在 Metamarkets 的初期，我们着力构建托管服务的业务端 Dashboard，允许用户随机查询和可视化事件流。数据仓库驱动着 Dashboard 的需求回到快速查询上来，数据可视化，以使数据可视化工作建立在为用户提供交互式体验上。

除了查询延迟的需求，该系统必须是多组织用户共享和高可用的。

Metamarkets 产品是用在高并发环境下。如果一个系统在软件升级或网络故障时不可用，那企业在宕机时间就要花费高昂的代价，以至于许多企业都不能承受等待恢复的过程。通常情况下，对于缺乏内部运营管理人员的初创型公司，宕机时间直接决定了业务的成败。

最后，Metamarkets 在其早期面临的另一项挑战是如何允许用户和报警系统能够“实时”做出业务决定。从一个事件被创建，到该事件能够被查询到的时间长度，决定了他们的系统在面对潜在的灾难性情况时的反应速度有多快。

像 Hadoop 这样的，主流的开源数据仓库系统都不能满足我们次秒级数据摄入延迟的需求。多个行业都存在数据挖掘，摄入和可用性跨度的问题。自 2012 年 10 月 Druid 开源以来，多个公司用它来部署一个视频、网络监控、运营监控和在线广告分析平台。

# 3.架构

Druid 集群由不同类型的节点组成，每个类型的节点被设计用来执行一组特定的任务。我们相信这种设计分离了关注点，同时简化了整个系统的复杂性。不同的节点类型操作相对独立，并且不同的节点类型之间仅有极少的交互。因此，集群内通讯故障对数据可用性的影响微乎其微。

为了解决复杂的数据分析问题，不同的节点类型组合在一起形成一个完成的工作体系。Druid 的名字来自于许多角色扮演游戏中存在的 Druid 类：它是一个变形人，可以以许多不同的外形出现在一个团队中。Druid 重点的数据构成和数据流向如 图 1 所示。

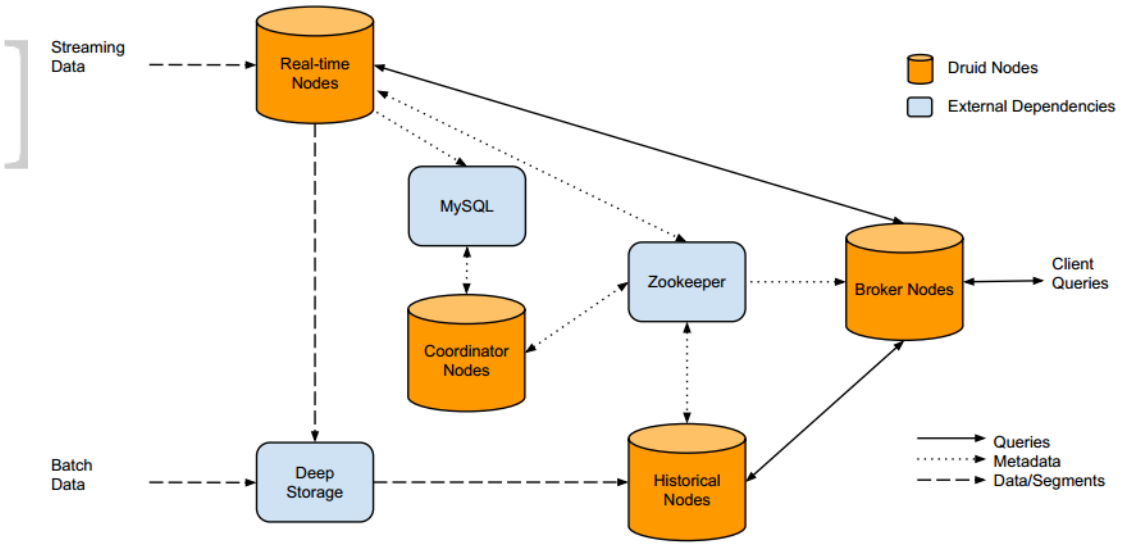



**图 1：Druid 集群和集群中的数据流概览**

## 3.1 Real-Time 节点

Real-Time 节点封装了摄入和查询事件流的功能。经过该节点的事件被实时索引，且对查询即时可用。节点只关注一段较短时间范围内的有效事件，并且定期地将其在这段较短时间范围内收集到的一批批不可变事件，传送给集群中专门处理批量不可变事件的其他节点。Real-Time 节点利用 Zookeeper[19]来跟集群中剩余的其它节点协调配合。所有节点在 ZooKeeper 中通知它们的在线状态和它们所提供的数据。

Real-Time 节点为所有将要流入的事件维护一个内存索引缓冲器(Index buffer)。 当有事件摄入，且索引可以被直接查询的时候，这些存在于缓冲器中的索引将会增量更新。对于事件查询，Druid 表现为一个存在于 JVM 基于堆的缓冲器当中的存储行。为避免堆溢出的问题，Real-Time 节点将周期性地，或者当达到某些最大行数限制后，持久化其存在于内存当中的索引到硬盘。这个持久化的过程将内存缓冲器(In-memory buffer)中的数据转换成为一个列存储格式(第 4 部分描述)。任何一个已被持久化了的索引都是固定不变的，为了能够使其仍然能够被查询，Real-Time 节点将已持久化了的索引装载到堆外内存(off-Heap memory)。该过程如图 2 所示，详细描述请参见[33]。

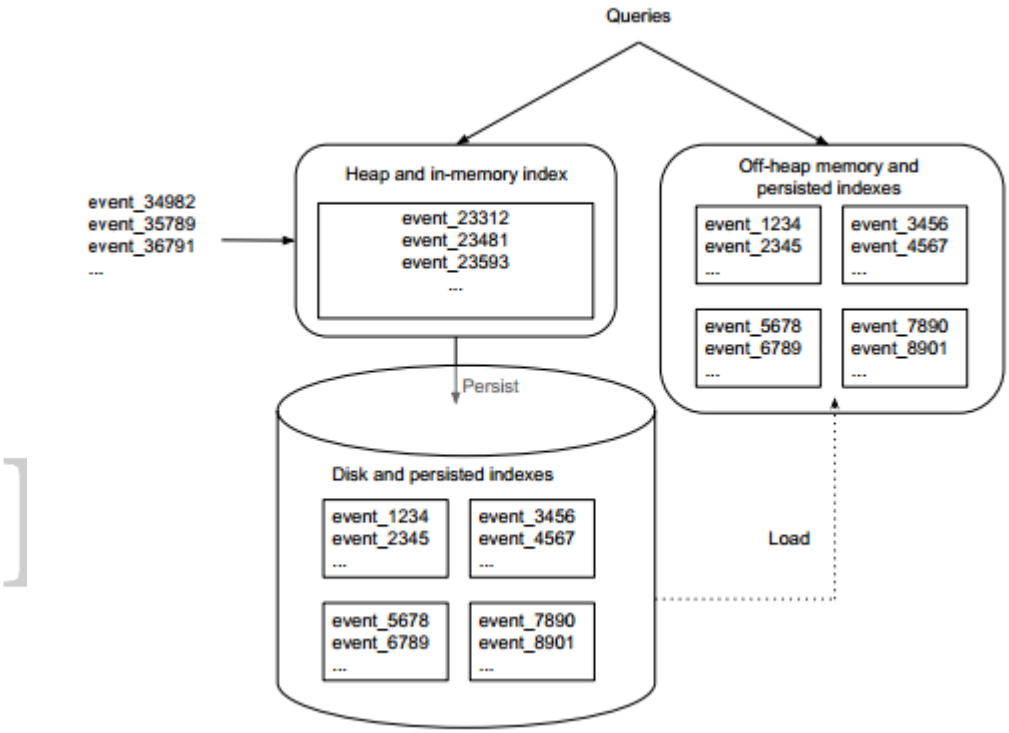



**图 2：Real-Time 节点将事件缓冲(buffer)到一个内存索引(in-memory index)当中,该索引将会被定期被持久化到硬盘。已持久化的索引将在传递出去之前被定期合并。查询将会同时命中内存中或者已持久化的索引。**

所有 Real-Time 节点将周期性地执行一个用以搜索所有本地已持久化的索引的后台任务。该任务合并上述这些索引，然后创建一个不可变数据块，该数据块包含着某个时间跨度范围内的所有已被 Real-Time 节点摄入的事件，我们称这样的一个数据块为一个“Segment”。在数据传输阶段，Real-Time 节点上传上述数据块(Segment)到永久备份存储中，通常是一个分布式文件系统，如 S3 [ 12 ]或 HDFS[ 36 ]。Druid 称这些永久数据备份存储为“Deep Storage”

数据的摄入，持久化，合并，和传输过程是流畅的；上述任何过程都没有数据丢失。

图 3 阐明了 Real-Time 节点的一系列操作。节点在 13:37 开始，并且只接收当前一小时和下个小时的事件。当事件被摄入后，节点通告其正在提供一个时间区间为 13:00 ~ 14:00 的数据块(Segment)。每 10 分钟(持久化周期，可配置)，节点会将其在内存中缓冲的数据持久化到硬盘，并刷新内存缓冲区。

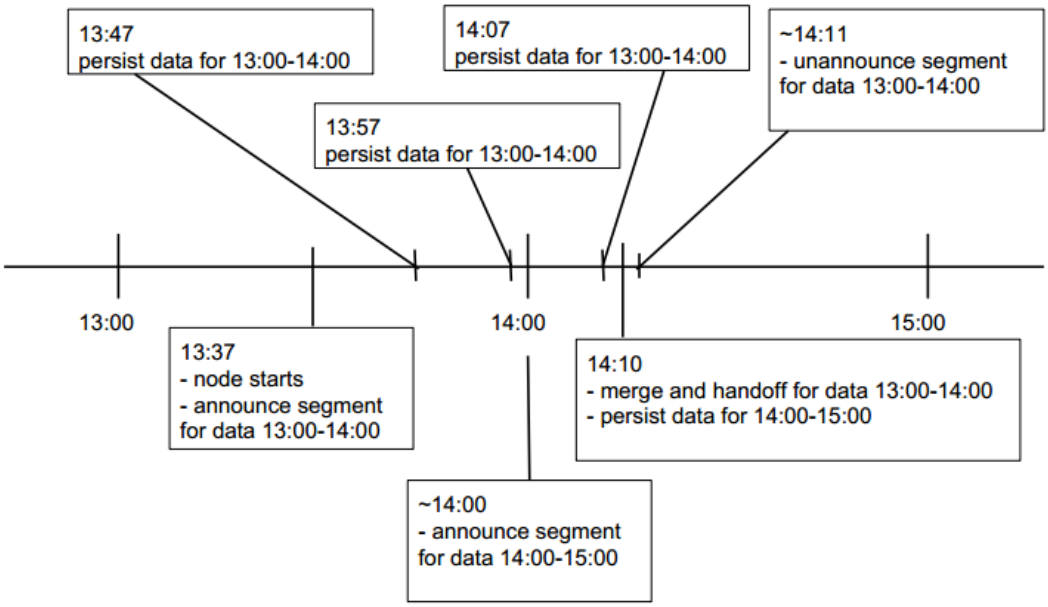



**图 3：节点启动，摄入数据，持久化，然后定期将数据传递出去。这一过程将无限重复。不同 Real-Time 节点运行的时间周期是可配置的。**

在这个小时临近结束的时候，节点可能会收到 14:00 ~ 15:00 之间的事件。

在这个时候，节点准备为下个小时提供数据，并创建一个新的内存索引(in-memory index)。紧接着，节点通告它也同时提供一个时间区间为 14:00 ~ 15:00 的数据块(Segment)。节点并没有立刻合并已持久化了的，在 13:00 ~ 14:00 的索引，而是等待一个可配置的窗口期(window period)，以保证 13:00 ~ 14:00 这个时间区间里延后的事件能够到达。这个窗口期使因事件传送延迟而造成的数据丢失的风险降到最低。在窗口期的末尾，节点合并所有已持久化了的 13:00 ~ 14:00 间的索引成为一个不可变数据块(Segment)，并将其分发出去。一旦该数据块(Segment)被成功加载，并且在 Druid 集群中的其他地方可查，Real-Time 节点就刷新其所有已收到的 13:00 ~ 14:00 间的数据相关信息，同时通告它正在提供这些数据。

### 3.1.1 可用性和可扩展性

Real-Time 节点是数据消费者，且需要有响应的生产者提供数据流。通常，如图 4 所示，为了满足数据持久化的要求，我们会在生产者和 Real-Time 节点之间部署像 Kafka[21]这样的消息总线。Real-Time 节点靠从消息总线中读取事件来摄入数据。通常情况下，从事件产生到事件消费的时间大约为几百毫秒。

图 4 中消息总线有两个目的。第一，消息总线作为流入事件的缓冲器。消息总线，如 Kafka，维护位置偏移用来标识一个消费者（Real-Time 节点）在事件流中读取了多远。消费者可以以编程的方式更新这些偏移。Real-Time 节点在每次持久化它们内存缓冲器中的数据到硬盘时更新该偏移量。在失败或者恢复的场景中，如果一个节点没有丢失硬盘，它可以重

新从硬盘加载所有已持久化了的所有索引，然后继续从该节点最后提交的偏移量开始读取事件。从最近提交的偏移量开始摄入事件极大地缩短了节点恢复的时间。实践中，我们发现节点在几秒钟之内就可以从失败场景中恢复。

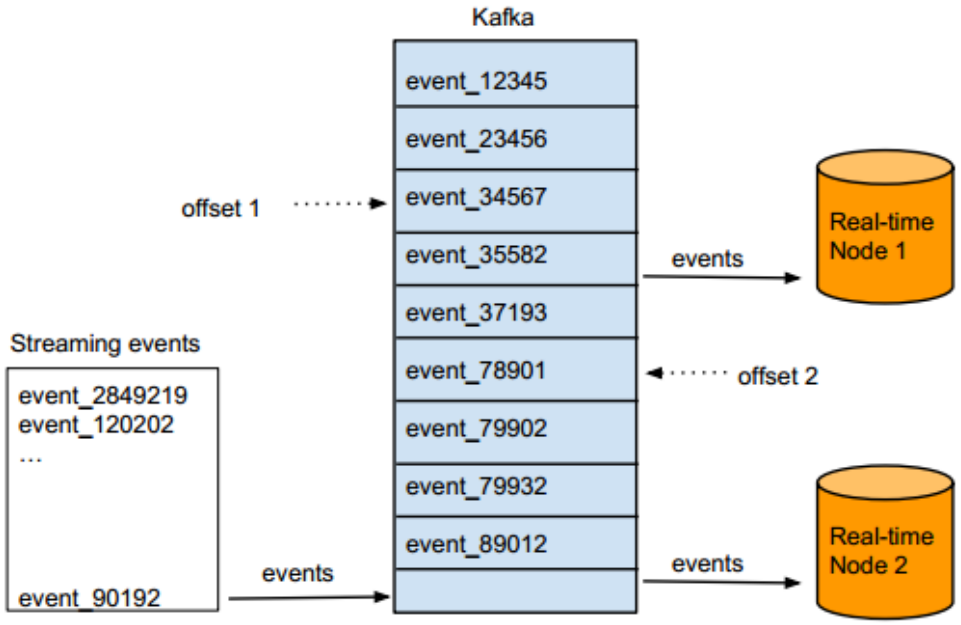



**图 4：多个 Real-Time 节点可以从同一个消息总线(message bus)中读取数据。每一个节点都各自维护其自己的偏移量(offset)。**

第二，消息总线的另一个作用是，可以让多个 Real-Time 节点从同一个端点读取事件。多个 Real-Time 节点可以从消息总线中摄入同一批事件，从而创建事件副本。在一个节点完全不可用，而且硬盘也丢失的情况下，已复制的事件流能确保不会丢失数据。一个单一的数据摄入端点也同样允许数据流分片，从而使多个 Real-Time 节点每次摄入一个数据分片。这允许额外的 Real-Time 节点可以无缝加入。在实践中，这种模型已经支撑了一个最大的 Druid 集群用于生产环境，该集群可以消费大约 500 MB/s (150,000 Events/s or 2 TB/hour)的原始数据。

Druid-China

## 3.2 Historical 节点

Historical 节点封装了加载和提供 Real-Time 节点创建的不可变数据块(Segment)的功能。在许多实际的工作流当中，Druid 集群中加载的大部分数据是不可变的，因此，Historical 节点通常是 Druid 集群的主要工作者。Historical 节点遵循无共享架构，所以，节点之间没有公共资源竞争。节点之间互不知晓，因此操作上较为简单。他们仅仅知道如何加载、删除和提供不可变数据块(Segment)的服务。

和 Real-Time 节点类似，Historical 节点在 Zookeeper 中通告他们的在线状态和他们提供的数据。加载和删除数据块(Segment)的指令通过 Zookeeper 发送，指令包含了数据块(Segment)在 Deep Storage 中的位置，以及如何解压和处理数据块(Segment)。在 Historical 节点从 Deep Storage 下载某个指定的数据块(Segment)之前，它首先检查本地缓存，缓存中维护了一组关于哪些数据块(Segment)已经存在于当前节点的信息。如果数据块(Segment)在本

地缓存中不存在，Historical 节点将会从 Deep Storage 中下载上述指定的数据块(Segment)。该过程如图 5 所示。一旦下载执行完成，数据块(Segment)的相关信息就会在 Zookeeper 中通告。此时，数据块(Segment)即可被查询。本地缓存也允许 Historical 节点快速更新和重启。在初始化的时候，节点检索其自身缓存的所有数据并立即提供数据服务。

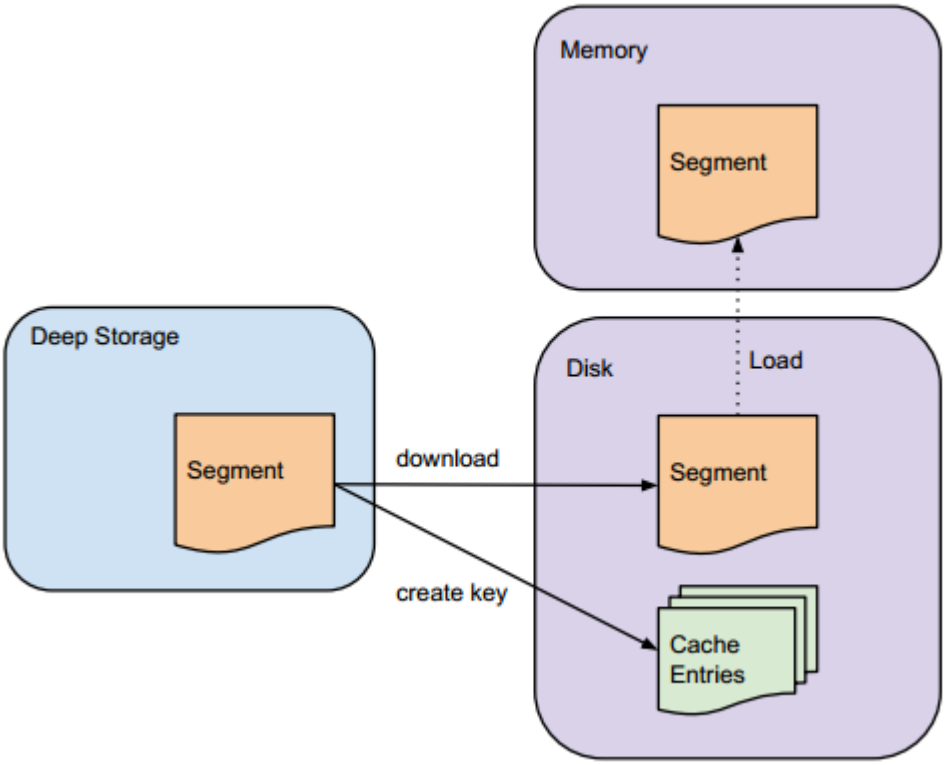



**图 5：Historical 节点从 Deep Storage 中下载不可变数据块(Segment)。数据块(Segment)必须在载入内存之后才能被查询到。**

Historical 节点可以支持一致性读(read consistency)因为它们仅处理不可变数据。不可变数据块也允许简单的并行化模型：Historical 节点可以并行地扫描和聚合不可变数据块，而不会被阻塞。

Druid-China

### 3.2.1 层

在不同的层，Historical 节点可以分为不同的组，在一个给定的层中，所有的节点都使用统一的配置。可以为每一层设置不同的性能和容错能力参数。

把节点分层的目的是为了能够根据数据块(Segment)的重要性，将数据设定为更高或者更低的优先级分发。举例来说，可以将拥有更多核心（core）和更大内存容量的 Historical 节点提升到“热数据”层(“hot” Tier)。热数据集群可以配置用以下载更经常被访问的数据。同样的，也可以用更少的强大硬件来创建冷数据集群（“cold” cluster）。冷数据集群中将紧紧含有那些不经常被访问的数据块(Segment)。

### 3.2.2 可用性

Historical 节点的数据块(Segment)加载和卸载指令依赖 Zookeeper。 假如 Zookeeper 变得不可用，Historical 节点就不能再提供新数据或者删除旧数据，然而，由于查询服务是基于 HTTP 协议的，Historical 节点仍然可以基于它们现有的数据为查询请求提供响应。 这意味着 Zookeeper 停机不影响 Historical 节点的现有数据可用性。

## 3.3 Broker 节点

Broker 节点作为 Historical 节点和 Real-Time 节点的查询路由器。

Broker 节点能够从 Zookeeper 中发布的元数据(Metadata)中知道：哪些数据块(Segment)是可查询的？以及这些数据的位置在哪？ Broker 节点将进来的查询请求路由到合适的 Historical 或者 Real-Time 节点上。 Broker 节点会先合并来自 Historical 节点和 Real-Time 节点的结果片段成为一个最终结果，然后返回给请求者。

### 3.3.1 缓存

Broker 节点维护一个 LRU [31, 20]失效策略缓存。上述缓存可以使用本地堆内存或者外部分布式 key/value 数据仓库，如 Memcached [16]。每一次一个 Broker 节点收到一个查询请求，它首先将该查询映射到一系列的数据块(Segment)。某些数据块(Segment)的结果可能已经存在于缓存之中，就没必要重新计算。对于那些缓存中没有的结果，Broker 节点将会转发上述查询到正确的 Historical 节点和 Real-Time 节点。一旦 Historical 节点返回其计算结果，Broker 节点将以每一个数据块(Segment)为基准缓存这些结果，以备后用。这一过程如图 6 所示。实时数据不会被缓存，因此对实时数据的请求总是被转发到 Real-Time 节点。实时数据一直在变化，所以缓存实时数据的计算结果没有可信性。

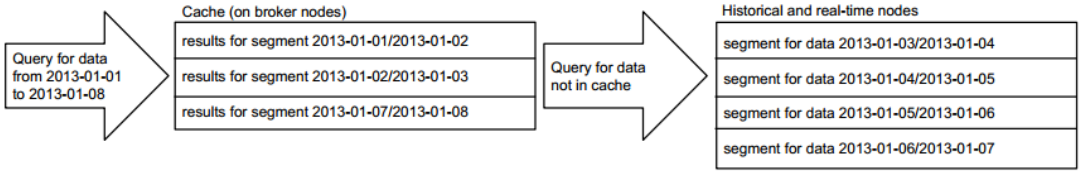



**图 6：每一个数据块(Segment)的结果都会被缓存。查询将缓存的结果与 Historical 节点和 Real-Time 节点上计算的结果合并。**

缓存也作为一个额外层次的数据可用性。万一所有的 Historical 节点都不可用，这些缓存下来的计算结果可能仍然可以响应查询。

### 3.3.2 可用性

假如所有的 Zookeeper 都停机，数据依然可查询。如果 Broker 节点无法和 Zookeeper 通信，它们使用其最后一次获知的集群视图(view of the cluster)继续转发查询到 Real-Time 和

Historical 节点。Broker 节点假定集群的结构和停机之前相同。实际上，这一可用性模型已经在我们诊断 Zookeeper 集群停机的相当长一段时间里，支撑我们的 Druid 集群持续提供查询服务。

## 3.4 Coordinator 节点

Druid Coordinator 节点主要负责 Historical 节点上的数据管理和分配。Coordinator 节点通知 Historical 节点加载新数据，删除旧数据，复制数据和移动数据以均衡负载。Druid 使用多版本并发控制交换协议来管理不可变数据块(Segment)，以保持视图的稳定性。如果任何不可变数据块(Segment)包含了已被新的数据块(Segment)完全废弃了的数据，旧的数据块(Segment)就会被从集群中删除。Coordinator 节点经过一个领导者选举(leader-election)过程确定一个唯一的节点来发挥协调者(Coordinator)的功能。剩余的 Coordinator 节点作为冗余备份。

Coordinator 节点定期运行，以确定集群的当前状态。Coordinator 节点通过比较集群的预期状态和当前实际运行时状态做决定。和所有 Druid 节点一样，Coordinator 节点通过维持一个和 Zookeeper 的连接来获取集群的当前信息。Coordinator 节点也维持一个和 MySQL 数据库的连接，数据库中包含有额外的操作参数和配置信息。存在于 MySQL 数据库中的其中一块关键信息，是一个包含应该由 Historical 节点提供的数据块(Segment)列表的数据库表(table)。该表可以被任何能创建数据块(Segment)的服务更新，比如 Real-Time 节点。MySQL 数据库也包含一个规则表(rule table),该表管理着在集群中，数据块(Segment)怎样被创建？如何被销毁？以及怎样被复制。

### 3.4.1 规则

规则管理着历史数据块(Segment)(Historical Segments)如何被加载和怎样从集群中删除。规则指明了数据块(Segment)应该如何被分配到不同的 Historical 节点层(Historical node Tier)，以及在每个层应该存在几个数据块(Segment)的副本。规则也可能指明了数据块(Segment)应该在何时从集群中完全销毁。规则通常被设置为一段时间。举例来说，一个用户可能使用规则来加载最近一个月的数据块(Segment)到热数据集群(“hot” cluster)，最近一年的数据块(Segment)到冷数据集群(“cold” cluster)，删除其他所有更老的数据块(Segment)。Coordinator 节点从 MySQL 数据库的规则表(rule table)中加载一系列规则。规则可能特定于某一个指定的数据源，和/或者一个默认的规则配置。Coordinator 节点将通过循环匹配所有有效的数据块(Segment)，为每一个数据块(Segment)匹配到适用于它的第一个规则。

### 3.4.2 负载均衡

在一个典型的生产环境中，查询常常会命中几十个甚至数百个数据块(Segment)。由于每一个 Historical 节点的资源都是有限的，数据块(Segment)在集群中必须是分布式的，以确保集群的负载不至于太不均衡。确定最佳的负载分配需要一些关于查询模式和速度的知识。通常，对于一个单一数据源来说，查询会覆盖最近的，跨越连续的时间区间的一些数据块(Segment)。一般情况下，访问较小数据块(Segment)的查询速度会较快。

这些查询模式间接地使得近期的历史数据块(Segment)复制更加快速，将在时间上相近

的大数据块(Segment)分发到不同的 Historical 节点上，共置来自于不同数据源的数据块(Segment)。为了能够以最佳的方式在集群间分发和均衡数据块(Segment)，我们开发了一个基于成本的(cost-based)优化过程，该过程考虑到了数据块(Segment)的数据源，近因(recency)和大小(size)。具体的算法细节超出了本文的讨论范围，可能在以后的文献中讨论。

### 3.4.3 复制

Coordinator 节点可能通知不同的 Historical 节点去加载同一个数据块(Segment)的一个复制副本。在历史数据计算集群的每一层(Tier)中，复制的数量是完全可配置的。设置高水平的容错能力，就可以设置相应多的副本数。被复制出来的数据块(Segment)副本会被当做原数据同等对待，也要遵循相同的负载分配算法。通过复制数据块(Segment)，单个 Historical 节点故障在 Druid 集群中是透明的。我们使用这一特性升级应用。我们可以无缝地将一个 Historical 节点摘除下线，升级，将其备份，然后对集群中的每一个 Historical 节点重复执行上述过程。在过去的两年里，我们 Druid 集群从来没有因为软件升级而停机。

### 3.4.4 可用性

Druid Coordinator 节点将 Zookeeper 和 MySQL 作为外部依赖。Coordinator 节点依赖 Zookeeper 来确定哪些 Historical 节点已经在集群中存在。如果 Zookeeper 变得不可用，Coordinator 将不能再发送指令来分配、均衡或删除数据块(Segment)。但是，这些操作丝毫不会影响到数据的可用性。

对 MySQL 的响应故障和 Zookeeper 不可用的设计原理是一样的：如果负责协调的外部依赖故障，则集群保持现状。Druid 使用 MySQL 来存储操作管理信息和应该存在于集群中的数据块(Segment)的元数据信息。如果 MySQL 宕机，这些信息对于 Coordinator 节点来说将变得不可用。但是，这并不意味着数据本身不可用。如果 Coordinator 节点不能和 MySQL 通信，它们将停止分配新的或删除旧的数据块(Segment)。在 MySQL 宕机期间，Broker, Historical, 和 Real-Time 节点仍然可供查询。

Druid-China

# 4.存储格式

Druid 中的数据表(被称为 数据源)是一系列时间戳标记的事件和被分片的一组数据块(Segment)的集合，在数据源中每一个数据块(Segment)通常包含 500-1000 万行数据。正式地，我们定义一个数据块(Segment)为跨越一段时间范围的一批数据行。数据块(Segment)代表了 Druid 中的基本存储单元，并且数据的复制和分配都是在数据块(Segment)层级上完成的。

Druid 将一个必选的 timestamp 列作为简化数据分配策略、数据保留策略和第一级查询的修剪的方法。Druid 根据其定义好的时间区间(time intervals)(通常是一小时或者一天)将数据分片，也可能进一步将其他列的值分割，以达到期望的数据块(Segment)大小。数据块(Segment)分片的时间粒度是一个数据量和时间范围的函数。分散在一年内的带有 timestamp 的数据集按天分区比较好；分散在一天内的带有 timestamp 的数据集按小时分区比较好。

数据块(Segment)由数据源标识符、数据的时间间隔 和 随数据创建而增加的版本号字符串来唯一标识。版本号字符串标识了数据块(Segment)的新旧程度；在一定的时间范围内，

带有较近版本号的数据块(Segment)，比带有较老版本号的数据块(Segment)拥有更新的数据视图(View Of Data)。这部分元数据被系统用于并行控制；对于一定时间范围内的数据块(Segment)，读操作总是根据这一时间范围的最近的版本标识符来访问数据。

Druid 数据块(Segment)被按列存储。我们已知的状况是，Druid 最好被用于聚合事件流(所有流入 Druid 的数据必须带有 timestamp)，将聚合信息按列存储，而不是按行存储的优势在于这样有更好的文档记录[1]。列式存储可使 CPU 的使用更加高效：只扫描和加载那些需要的数据。在行式数据仓库中，所有与行相关的列必须作为聚合的一部分被扫描。额外的扫描时间会导致性能显著下降[1]。

Druid 拥有多个列类型来表示各种数据格式。根据列类型，使用不同的压缩方法来减少一列在内存和硬盘上的存储开销。在我们给出的表 1 的例子当中，page, user, gender, 和 city 列仅仅包含字符串。直接存储字符串会带来不必要的昂贵开销，所以字符串列可以以字典编码。字典编码是一种常见的数据压缩方式，这种压缩方式已经被用于其他数据仓库，如 PowerDrill [17]。在表 1 的例子当中，我们可以将每一页(page)映射到一个唯一的整数标识符。

Justin Bieber -> 0

Ke$ha -> 1

该映射允许我们将 page 的列表示为一个整数数组，数组的索引和原始数据集的行一致。对于 page 列，我们可以用如下的方式来唯一标识一个 page:

[0, 0, 1, 1]

产生的整数数组给其自身提供了非常好的数据压缩方法。基于编码的通用压缩算法在列式存储中相当普遍。Druid 使用 LZF[24]压缩算法。

相似的数据压缩算法可以被应用到数字列(Numeric Column)上。举例来说，表 1 中字符增加和字符删除的列也可以被压缩成为单个数组。

Characters Added -> [1800, 2912, 1953, 3194]

Characters Removed -> [25, 42, 17, 170]

在这个例子中，我们按照其字典表示相反的方式压缩原始值。

## 4.1 数据过滤索引

Druid-China

在许多真实的 OLAP 工作流当中，查询的目标是一些指标的聚合结果，这些聚合结果由那些能满足规范的维度聚合而来。一个示例查询如：“Wikipedia 被来自于 San Francisco 的男性编辑了多少次？”，这个查询基于一个布尔类型的维度值的表达式过滤了 Wikipedia 数据集。在许多现实世界的数据集里，维度列包含字符串，指标列包含数值。Druid 为字符串列创建了额外的查询索引，以至于只有跟指定查询过滤器相关的那些数据行才会被扫描。

我们回想下表 1 中的 page 列。对于表 1 中的每一个唯一列，我们可以对看到的表里面每一行的某个特定的 page 构建一些表示方法。我们可以将这些信息存储在一个二进制数组，该数组的索引代表我们的行。如果我们在某一行里面看到一个特定 page，那个数组索引就会被标记为 1。例如：

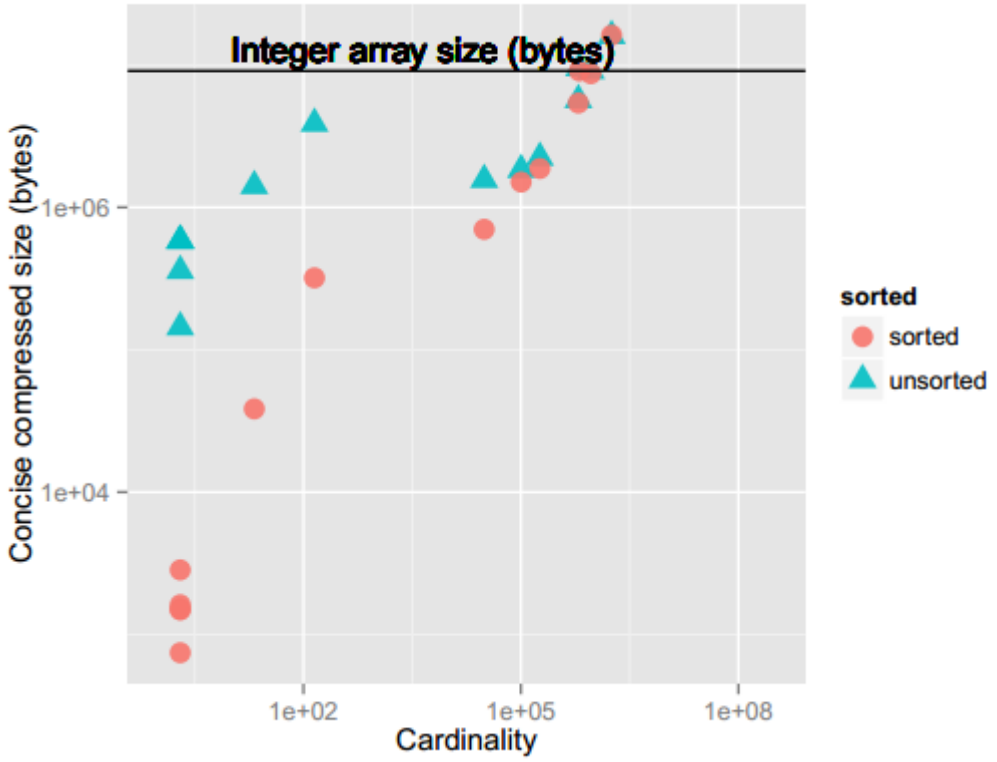
Justin Bieber -> rows [0, 1] -> [1][1][0][0]

Ke$ha -> rows [2, 3] -> [0][0][1][1]

我们看到第 0 行和第 1 行都有 Justin Bieber。这种列值到行索引的映射形成了一个倒排索引[39]。要知道哪些数据行里包含有 Justin Bieber 或者 Ke$ha，我们可以将两个数组进行 OR 运算。

[0][1][0][1] OR [1][0][1][0] = [1][1][1][1]

这种在大型位图集合(Bitmap sets)上执行布尔运算的方法常被用于搜索引擎当中。关于 OLAP 工作负载的位图索引(Bitmap indices)的细节描述请参见[32]。位图压缩算法(Bitmap Compression Algorithms)是一个明确定义的研究领域[2, 44, 42]，通常利用游程长度编码(run-length encoding)。Druid 选择使用简洁的算法[10]。图 7 展示了使用简明压缩算法(Concise algorithm)与使用一个整数数组的字节数对比。结果是在一个单线程，2G 堆(Heap),512M 新生代(young gen)，和每次运行间强制垃圾回收(GC)的 cc2.8xlarge 系统上产生的。数据集是来自于 Twitter garden hose[41]数据流一天的数据收集。该数据集包含有 2,272,295 行，12 个不同基数的维度。作为一个额外的比较，我们也最大化压缩数据行。




Druid China

**图 7：整数数组和简单集合的大小对比**

在未排序的情况下，总的简明数据大小是 53,451,144 字节，总的整数数组大小是 127,248,520 字节。总体来说，经过简明压缩的数据集比整数数组减小了大约 42%。在排完序的情况下，简明压缩后总的数据大小是 43,832,884 字节，总的整数数组的大小是 127,248,520 字节。我们注意到，有趣的是排序后，全局压缩程度几乎没有增加。

## 4.2 存储引擎

Druid 的持久化组件允许不同的存储引擎接入，比如 Dynamo [12]。这些存储引擎可能将数据存储在，像 JVM 堆(JVM Heap)这样的，全内存结构 或者 内存映射结构(in memory-mapped structures)中。切换存储引擎的能力使得 Druid 能够根据某个特定应用的规格来配置。一个基于全内存的存储引擎可能比一个基于内存映射的存储引擎在操作上要花费更大的代价，但是如果更加关注性能，前者可能是更好的选择。Druid 默认使用基于内存映射的存储引擎。

当使用基于内存映射的存储引擎的时候，Druid 依赖操作系统来给内存内外的数据块(Segment)分页。假定仅有被加载到内存中的数据块(Segment)能被扫描，基于内存映射的存储引擎允许将最近的数据块(Segment)保存在内存页中，然而那些从不被用到的数据块(Segment)会被置换出去。使用基于内存的存储引擎的主要缺点是：当查询需要比一个给定节点的存储能力更多的数据块(Segment)载入内存页的时候，查询性能将面临数据块(Segment)出入内存页的开销问题。

# 5.查询 API

Druid 有他自己的查询语言，接受 POST 请求作为查询。Broker, Historical, 和 Real-Time 节点共同使用相同的查询 API。

POST 请求的 body 体是一个包含键值对(key-value pairs)JSON 对象，键值对指定了不同的查询参数。一个典型的查询将需要含有数据源(DataSource)的名称，结果数据的粒度(granularity)，目标的时间范围，查询类型和将要聚合的指标(Metrics)。返回结果也是一个包含有指定时间范围内的聚合后的指标的 JSON 对象。

大多数查询类型也会支持过滤器。一个过滤器是一个布尔类型的维度名和值的键值对表达式。可以指定任何数字和维度组合。当提供一个过滤器，仅有属于该过滤器的子数据集才能被扫描。处理复杂嵌套过滤器的能力使得 Druid 能够钻取任意深度的数据集。

具体的查询语法取决于查询类型和查询的目标信息。例如，对一周范围内的 count 查询如下：

```
{
    "queryType": "timeseries",
    "dataSource": "wikipedia",
    "intervals": "2013-01-01/2013-01-08",
    "filter": {
        "type": "selector",
        "Dimension": "page",
        "value": "Ke$ha"
    },
    "granularity": "day",
    "aggregations": [{"type": "count", "name": "rows"}]
}
```

Druid-China

上述查询将会返回 Wikipedia 数据源中从 2013-01-01 到 2013-01-08 期间数据行的计数，查询过滤了这些数据中“page”的值等于“Ke$ha”的数据。如下，结果将会被按天分组，并将以 JSON 数组的方式返回：

```
[{
    "timestamp": "2012-01-01T00:00:00.000Z",
    "result": {
        "rows": 393298
    }
},
{
    "timestamp": "2012-01-02T00:00:00.000Z",
```

```
        "result": {
            "rows": 382932
        }
    },
    ...
    {
        "timestamp": "2012-01-07T00:00:00.000Z",
        "result": {
            "rows": 1337
        }
    }]
```


Druid 支持很多类型的聚合，包括对浮点型、整型数值求和，求最小值，求最大值，还有像求基数预估和近似分位预估这样的复杂聚合。聚合的结果可以再按数学表达式结合成为其他聚合。详尽描述查询 API 已经超出了本文的范围，可在线获取更多信息 3。

在撰写本文时, Druid 还未实现 Join 查询。这也是工程资源分配和用例决策的一项职能，而不仅仅是完全由技术水平来驱动。事实上，Druid 的存储格式也允许 Join 查询(也不违以维度形式组织的列式原则)的实现，而且我们每隔几个月也就关于 Join 查询的实现展开会话交流。到目前为止，我们认为就实现的代价而言，还不值得我们团队投入。这个决定的原因有两个。

1. 以我们的经验而言，扩展 Join 查询已经成为分布式数据库工作中的一个常规瓶颈。
2. 和通过管理高并发来预见问题的收益相比，Join 查询会加重工作负载，被认为是没有多大意义的。

从本质上来讲，Join 查询就是根据一个共享的键的集合来合并两个或者更多的数据流。对于 Join 查询，我们意识到，高级别的主要策略是 Hash 策略或者排序-合并策略。Hash 策略要求所有的，而不是只有一个数据集可用，当某些东西看起来像 Hash 表的时候，然后在“Primary”流中。基于这个 Hash 表的每一行，一个查找操作被执行，排序-合并策略假设每一个流都是已经按 Join 的键排好序的，这样就使得流的增量的 join 操作是可以执行的。但是这些策略中的每一个策略，都需要一定数量的具体的流，这些流要么是按照排好的序，要么是 Hash 表的形式。

当 Join 查询的所有参与方的大表(多于 10 亿条记录)形成一个预 join 流的时候，就需要复杂的分布式内存管理。内存管理的复杂程度只会被我们将要处理的高并发，多租户的工作负载放大。这是我们所知道的一个活跃的学术研究课题，我们也愿意用一种可伸缩的方式帮助解决这一问题。

# 6.性能

Druid 被数个公司用于生产当中，为了证实它的性能，我们选取了 2014 年早期的时候，Metamarkets 在主要生产环境运行的集群的一些真实的数字来做分享。为了和其他的数据库做比较，我们也把基于 TPC-H 合成负载的基准测试数据结果也罗列了出来。

## 6.1 生产中的查询性能

Druid 的查询性能会根据提交的查询的不同表现出显著的差异。例如，在某一时间范围内，根据一个给定的度量指标对拥有很大基数的维度值进行排序，要比一个的简单 count 查询性能差得多。如表 2 所示，我们选取了查询次数最多的数据源(DataSource)当中的其中 8 个来展示生产环境中 Druid 集群的平均查询延迟。

| Data Source | Dimensions | Metrics |
|---|---|---|
| a | 25 | 21 |
| b | 30 | 26 |
| c | 71 | 35 |
| d | 60 | 19 |
| e | 29 | 8 |
| f | 30 | 16 |
| g | 26 | 18 |
| h | 78 | 14 |

**表 2：生产环境数据源特征**

大约 30%的查询是涉及不同类型的指标和过滤器的标准聚合查询，60%的查询是对一个或者多个维度(Dimension)的 Group By 聚合查询，还有 10%的查询是 Search 查询和元数据(Metadata)获取查询。在聚合查询中，扫描的列(Column)数大致遵循一个指数分布。查询仅涉及一个单独的列(Column)的情况是很常见的，而涉及所有列(Column)的查询非常罕见。

关于查询结果的一些说明：

- 查询结果出自我们生产集群中的“热数据”层。在该层中，大约有 50 个数据源(DataSource)，有数百个用户提交查询。
- 在“热数据”层，大约有 10.5TB 可用内存(RAM)，约 10TB 左右的数据块(Segment)被载入。在该层中，共有大约 500 亿个 Druid 数据行。没有完全展示所有数据源(DataSource)的结果。
- "热数据"层使用 Intel® Xeon® E5-2670 处理器，由 672 个(超线程)核心(Core)和 1302 个处理线程组成。
- 使用了一个内存映射存储引擎(该机器被配置为内存映射数据，而不是加载数据到 Java 堆(Heap)。)

查询延迟见图 8，每分钟查询数见图 9。在所有的数据源(DataSource)当中，平均查询延迟大约为 550 毫秒，其中大约 90%的查询 1 秒内返回，95%的查询 2 秒内返回，99%的查询都能在 10 秒内返回。我们会不定期观察延迟峰值，如 2 月 19 日观察到的情况，我们其中一个最大的数据源(DataSource)上很高的查询负载加剧了 Memcached 实例的网络问题。

Druid-China

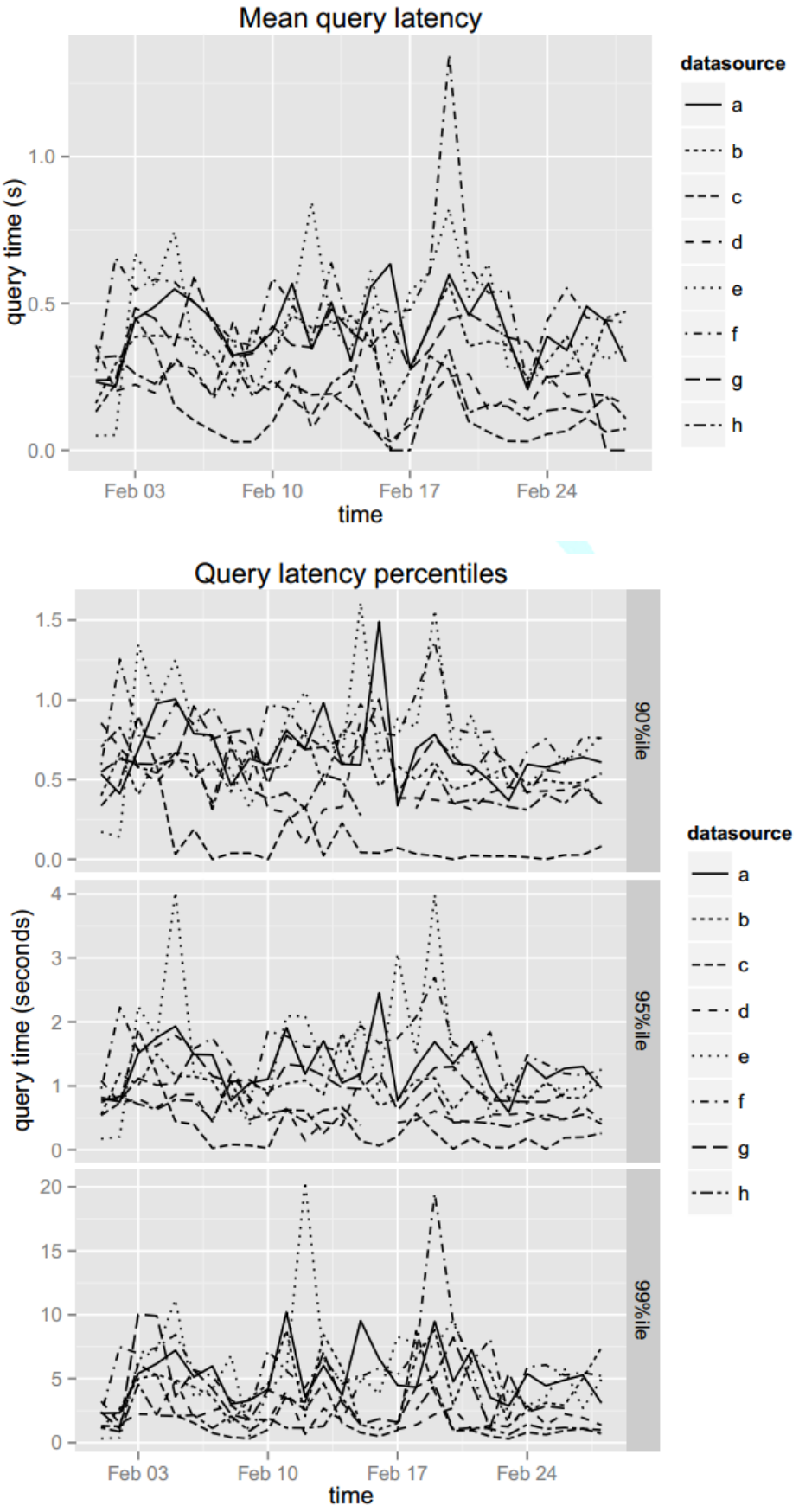



图 8：生产环境数据源(DataSource)的查询延迟。

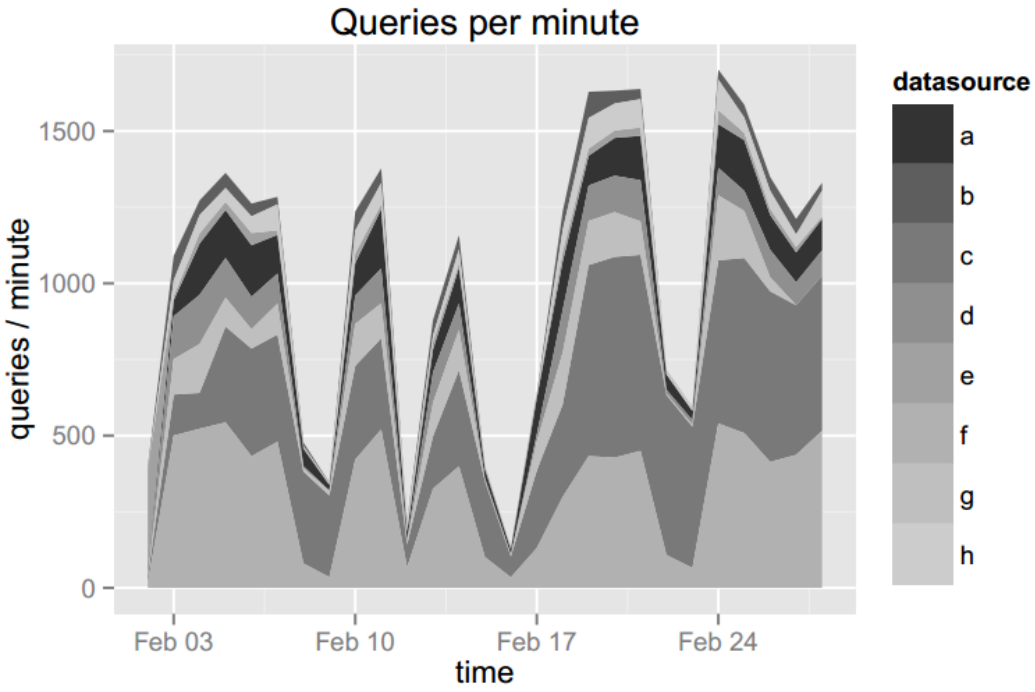



**图 9：生产环境数据源(DataSource)每分钟的查询数。**

## 6.2 基于 TPC-H 数据的基准查询测试

我们也呈现了基于 TPC-H 数据对 Druid 的基准测试。大部分 TPC-H 查询不能直接用于 Druid，因此我们选择了更加典型的 Druid 工作负载查询来演示查询的性能。作为对比，我们也提供了在使用 MyISAM 存储引擎的 MySQL 下执行同样的查询的结果。

我们之所以选择 MySQL 作为基准测试的对比对象，是因为它的普遍流行。我们不选择其他开源的列式数据仓库的原因是我们没有信心能将其调整到性能最佳状态。

我们的 Druid 安装部署使用了亚马逊弹性云计算(Amazon EC2)，Historical 节点使用了 m3.2xlarge 类型实例(Intel®Xeon®E5-2680 v2 @ 2.80GHz)，Broker 节点使用了 andc3.2xlarge 类型实例(Intel®Xeon®E5-2670 v2 @ 2.50GHz)。我们的 MySQL 安装部署同样是一个运行在 m3.2xlarge 类型实例之上的亚马逊远程数据服务(Amazon RDS)实例。

如图，图 10 是 1GB 的 TPC-H 基准数据测试结果，图 11 是 100GB 数据集的测试结果。我们用基准测试数据测试出：对给定的一段时间区间内，select count(*)的等价查询，Druid 的数据扫描速率为 53,539,211 行/秒/核心(rows/second/core)；一个 select sum(float)类型查询扫描速率为 36,246,530 行/秒/核心(rows/second/core)。

Druid China

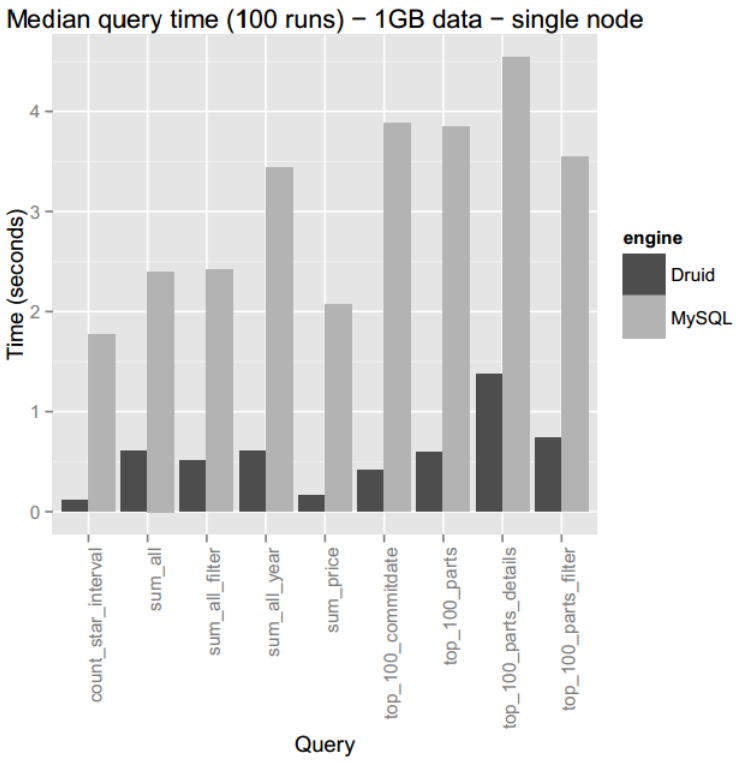



图 10：Druid & MySQL 基准测试 - 1GB 的 TPC-H 基准测试数据。

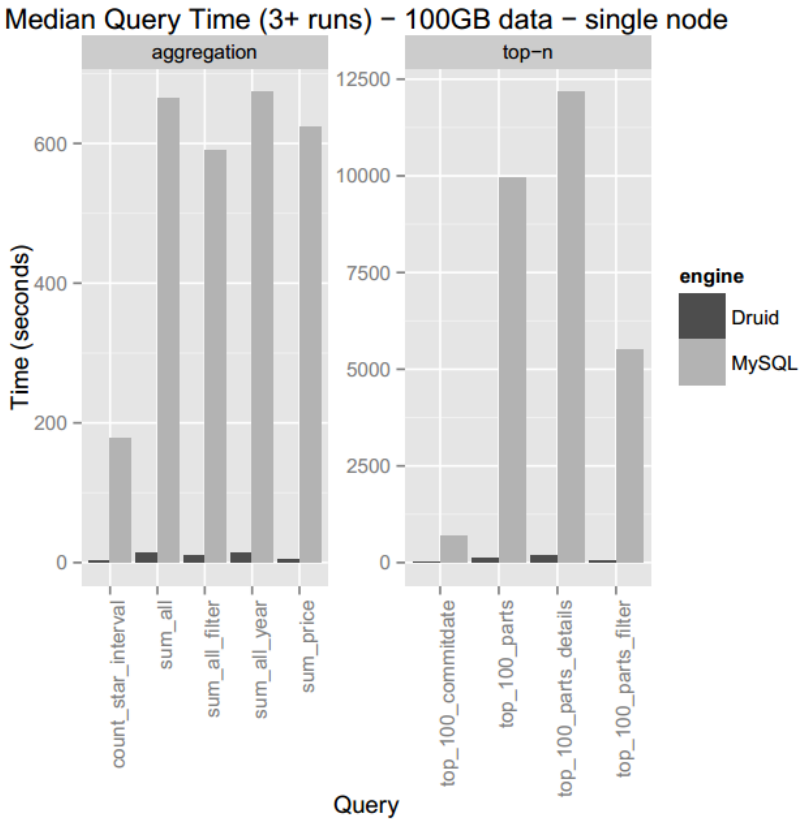



图 10：Druid & MySQL 基准测试 - 100GB 的 TPC-H 基准测试数据。

最后，为了能够满足数据量的增加的需求，我们将会用 100GB 的 TPC-H 基准测试数据集展示 Druid 的可扩展性。如图 12 所示，我们看到：当我们将核心(core)数从 8 核增加到 48 核的时候，并不是所有类型的查询都能够实现线性扩展，但是较简单的聚合查询能够实现线性扩展。

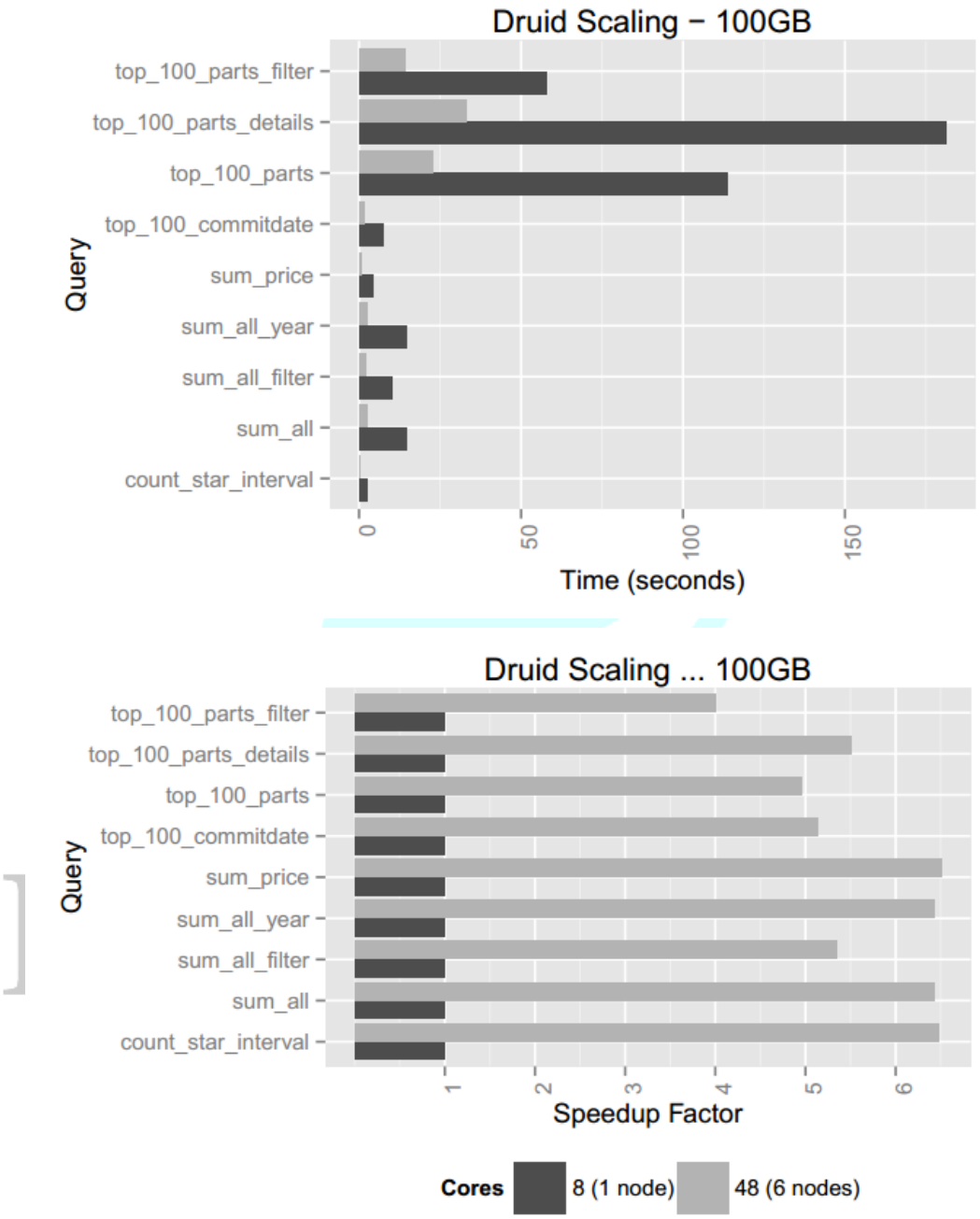



**图 12：Druid 扩展的基准测试 -- 100GB 的 TPC-H 基准测试数据。**

并行计算系统的速度增长往往受限于系统的顺序操作所需要的时间。在这种情形下，查询需要大量 Broker 级别的工作，而且这些工作都是非并行化的。

## 6.3 数据摄入(Ingestion)性能

为了展现 Druid 的数据摄入延迟，我们选取了生产环境下不同维度(Dimension)，不同度量指标(Metric)，不同事件集合(Event Volume)的几个数据源(DataSource)。我们的生产环境安装部署的数据摄入(Ingestion)由共计 360GB 内存(RAM)、96 个核心(Core)(12 x Intel®Xeon®E5-2670)的 6 个节点组成。

| Data Source | Dimensions | Metrics | Peak Events/s |
|---|---|---|---|
| s | 7 | 2 | 28334.6 |
| t | 10 | 7 | 68808.7 |
| u | 5 | 1 | 49933.93 |
| v | 30 | 10 | 22240.45 |
| w | 35 | 14 | 135763.17 |
| x | 28 | 6 | 46525.85 |
| y | 33 | 24 | 162462.41 |
| z | 33 | 24 | 95747.74 |

**表 3：不同数据源的数据摄入特征**

注意在该组织布局(Setup)中,其他几个数据源(DataSource)正在被摄入,且许多其他的 Druid 相关数据摄入(Ingestion)任务在这些机器上同时并行运行。

Druid 的数据摄入(Ingestion)延迟高度依赖于正在被摄入的数据集的复杂度。数据的复杂度由每个事件(Event)的维度(Dimension)数、度量指标(Metric)数和我们想要对这些度量指标(Metric)执行的聚合类型决定。对于大多数这些基本数据集(仅有一个 timestamp 列)，我们的组织布局(Setup)可以以 800,000 事件/秒/核心（event/second/core）的速率摄入数据，这其实仅仅是对我们到底可以以多快的速度反序列化的一个量度(Measurement)。现实当中的数据集从来不会这么简单。表 3 展示了选取的几个数据源(DataSource)和它们数据摄入的特征。

基于表 3 的描述，我们可以看到数据摄入延迟的显著变化，且数据摄入(Ingestion)延迟的影响因素并不是总是维度(Dimension)数。结果如图 13 所示，我们看到，一些简单数据集的延迟较低，因为这些数据对应的生产者(producer)传输数据的速度较快。

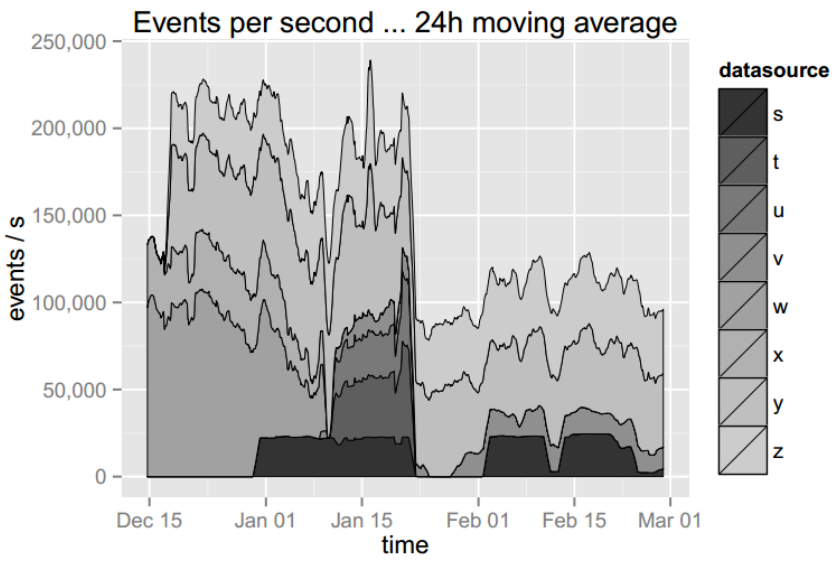



**图 13：集群数据摄入速度对比。**

我们将吞吐量定义为一个 Real-Time 节点可以摄入的事件的数量我们将一个 Real-Time 节点可以摄入事件,同时使其可查询，的数量定义为吞吐量。如果发送到 Real-Time 节点的事件太多，这些事件将被锁定，直到 Real-Time 节点有能力接收这些事件。在生产环境当中，我们对一个运行在亚马逊 cc2.8xlarge 实例上的，包含有 30 个维度(Dimension)和 19 个度量指标(Metric)的数据源(DataSource)测得的数据摄入(Ingestion)延迟的峰值是 22914.43 事件/秒/核心(event/second/core)。

我们提出的对延迟的测量值足以达到交互性所阐明的各种问题的要求。我们希望延迟的可变性少一些。通过增加更多硬件的方式来减少延迟仍然是可能的，但是我们并没有那么做，因为对我们来说，基础设施的成本仍然是我们需要考虑的问题。

# 7.生产环境中的 Druid。

在过去的几年中，关于如何用 Druid 处理生产上的工作负载，我们已经获得了丰富的知识，并且有了几个有趣的观点。

## 查询模式

Druid 经常被用于数据探索和数据报表生成。对于单个用户来说，使用数据探索的查询次数要远远多于使用数据报表查询的次数。为了在同一时间范围内缩小查找范围，探索查询常常会不断增加过滤器。用户倾向于探索内短时间范围内的近期数据。在生成数据报表的用例中，用户要查询的数据对应的时间跨度要长得多，但是这些查询通常很少而且大多是预设的。

## 多租户

在多租户的环境当中，成本高昂的并发查询可能是有问题的。对于大数据源(DataSource)的查询可能最终会命中集群中的所有 Historical 节点，消耗掉所有的集群资源。在这种情况系下，较小的，成本较低的查询可能会被锁定。我们引入了对查询的优化来解决这些问题。每一个 Historical 节点可以优先选取它们需要扫描的数据块(Segment)。适当的查询规划对于生产环境的工作负载至关重要。庆幸的是，对大量数据的查询趋向于报表生成的用例，可以降低优先级。当用户探索数据的时候，他们希望有不同的交互级别。

## 节点失效

在分布式环境下，单个节点失效很常见，但是很多节点同时失效并不常见。如果 Historical 节点完全失效且未恢复，它们的数据块(Segment)需要重新分配，这就意味着我们需要额外的集群容量来加载这些数据。任何时候添加额外的集群容量都会增加集群运行的成本。从我们的经验来看，很少看到有 2 个以上的节点同时失效，因此我们可以在集群中留出足够的容量来从 2 个 Historical 几点恢复数据。

Druid China

# 数据中心宕机

集群完全失效是有可能的，但这种情况极其罕见。如果 Druid 仅仅部署在单一的数据中心，将有可能出现整个数据中心失效。在这种场景下，将需要提供新的机器。只要 Deep Storage 依然可用，集群恢复的时间就取决于网络，因为 Historical 节点仅需要从 Deep Storage 重新下载所有的数据块(Segment)。在过去我们曾经经历过这样的失效，在 AWS 生态系统中恢复几个 TB 的数据需要几个小时的时间。

## 7.1 运行监控

对于一个大规模的分布式集群，适当的监控是至关重要的。在设计上，每一个 Druid 节点都会定期地发送出一系列运行指标。这些指标可能包括系统级别的统计数据，如 CPU 使用情况、可用内存和磁盘容量；JVM 统计信息，如垃圾回收时间、堆使用情况；或者是节点的具体指标，如数据块(Segment)扫描时间、缓存命中率和数据摄入(Ingestion)延迟。Druid 也发送出每一个查询的指标。

我们从一个生产环境下的 Druid 集群中发送出指标，并将这些信息加载到一个专用的 Druid 指标集群。 Druid 指标集群被用于探索生产集群的性能和稳定性。 这些专用的指标集群已经让我们找到了很多生产环境下的问题，比如渐进式查询速度分散(gradual query speed degregations)、硬件调优不充分和其他各种系统瓶颈。 我们也使用指标集群来分析人们都在生产环境中做了哪些查询，以及用户对哪些方面的数据感兴趣。

## 7.2 将 Druid 和流式处理程序相结合

现在，Druid 仅可以识别完全规范化的数据流。为了在生产中提供全面的商业逻辑支持，Druid 可以和一个像 Apache Storm [27]这样的流式处理程序相结合。

Storm 拓扑结构从数据流中消费事件(Event)，仅仅保留应时(on-time)的这些事件，并将其应用于相关的商业逻辑。 这可以从简单转换——如从 id 到 name 的查找，到复杂的操作——如多串流连接(multi-stream Join)。 Storm 拓扑结构将处理后的事件流以实时的方式转发到 Druid。 Storm 执行数据流处理工作，Durid 用于对实时和历史数据的查询响应。

## 7.3 多数据中心分布

大规模的生产环境宕机可能不仅影响单个节点，而且影响整个数据中心。 Druid 的 Coordinator 节点的层(Tier)配置允许数据块(Segment)在多个层(Tier)之间进行复制。 因此，数据块(Segment)可以在多个数据中心的 Historical 节点间完全复制。类似地，查询偏好(query preference)可以被分配到不同的层(Tier)。 可以将一个数据中心作为主集群(primary cluster)(同时接收所有查询)，另一个数据中心作为冗余集群(redundant cluster)。 如果一个数据中心离用户更近，就可以设计一个这样的组织布局(Setup)。

Druid-China

# 8.相关工作

Cattell [6]中对现存的可扩展 SQL(Scalable SQL)和 NoSQL 数据仓库做了很好的概述。Hu [18]献出了另一个关于流媒体数据库的优秀概述。Druid 的特点在某种意义上介于 Google 的 Dremel [28]和 PowerDrill [17]之间。Druid 具有 Dremel 已实现的大部分特点(Dremel 处理任意嵌套的数据结构，但是 Druid 仅仅只允许单一层面的基于数组的嵌套)，同时也包含了 PowerDrill 中提到的很多有意思的压缩算法。

尽管 Druid 和其他分布式列式数据仓库[15]建立在许多相同的原则之上，但是其他这些数据仓库都被设计为更为通用的 Key-Value 数据仓库[23]，并不支持存储级别的直接计算。对于 Druid 想要解决的那些问题，也有其他一些数据仓库被设计用来解决其中一些相同的问题。这些系统包括像 SAP 的 HANA [14] 和 VoltDB [43]这样的内存数据库。这些数据仓库缺乏 Druid 的低延迟数据摄入的特性。类似于 ParAccel [34]，Druid 也同时拥有原生的分析特性，但是 Druid 允许全系统无宕机地滚动式升级。

Druid 有两个子系统，一个是 Historical 节点内的读优化子系统，另一个是 Real-Time 节点中的写优化子系统，这与 C-Store [38]和 LazyBase [8]比较相似。Real-Time 节点被设计为摄入大容量的附加重数据，且不支持数据更新。不像前面所述的两个系统，Druid 的设计目标是支持 OLAP 事务，而不是 OLTP 事务。

Druid 在数据摄入上低延迟的特性与 Trident/Storm [27]和 Spark Streaming [45]具有相似之处，但是这两个系统都是专注于流处理上，而 Druid 是专注于数据摄入(Ingestion)和聚合(aggregation)。对于数据流入 Druid 之前的预处理(pre-processing)，流处理系统是对 Druid 很好的补充。

有一类的系统专门在集群计算框架之上处理查询。Shark [13]就是在 Spark 之上处理查询的这样的一个系统，Cloudera 的 Impala [9]是另一个专注于在 HDFS 上优化查询性能的系统。Druid 的 Historical 节点将数据下载到本地，并且只使用 Druid 的本地索引。我们相信这样的组织形式(setup)会带来更快的查询响应(更低的查询延迟)。

Druid 在其架构体系内使用一个独特的组合算法。我们相信，尽管没有其他数据仓库和 Druid 拥有相同的功能组合，但是其他数据仓库[26]同样也会使用和 Druid 一样的优化技巧，如使用倒排索引来执行快速过滤。

Druid-China

# 9.总结

在本文中，我们介绍了 Druid，一个面向列的分布式实时分析数据仓库。Druid 被设计为能够支持高性能的应用程序，被优化为能够拥有低延迟的查询。Druid 支持流式数据摄入(Streaming Data Ingestion)同时具有容错(Fault-tolerant)能力。我们还讨论了 Druid 的基准测试，并总结了诸如数据存储格式、查询语言和常规执行等关键架构相关的方面。

# 10.致谢

如果没有来自 Metamarkets 和社区当中的许多伟大工程师的帮助，Druid 不可能完成创建。我们想要感谢所有那些曾经为 Druid 代码库贡献过力量和提供过宝贵支持的人们。

# 11.参考文献

[1] D. J. Abadi, S. R. Madden, and N. Hachem. Column-stores vs. row-stores: How different are they really? In Proceedings of the 2008 ACM SIGMOD international conference on Management of data, pages 967–980. ACM, 2008.

[2] G. Antoshenkov. Byte-aligned Bitmap compression. In Data Compression Conference, 1995. DCC'95. Proceedings, page 476. IEEE, 1995.

[3] Apache. Apache solr. http://lucene.apache.org/solr/, February 2013.

[4] S. Banon. Elasticsearch. http://www.elasticseach.com/, July 2013.

[5] C. Bear, A. Lamb, and N. Tran. The vertica database: Sql rdbms for managing big data. In Proceedings of the 2012 workshop on Management of big data systems, pages 37–38. ACM, 2012.

[6] R. Cattell. Scalable sql and nosql data stores. ACM SIGMOD Record, 39(4):12–27, 2011.

[7] F. Chang, J. Dean, S. Ghemawat, W. C. Hsieh, D. A. Wallach, M. Burrows, T. Chandra, A. Fikes, and R. E. Gruber. Bigtable: A distributed storage system for structured data. ACM Transactions on Computer Systems (TOCS), 26(2):4, 2008.

[8] J. Cipar, G. Ganger, K. Keeton, C. B. Morrey III, C. A. Soules, and A. Veitch. Lazybase: trading freshness for performance in a scalable database. In Proceedings of the 7th ACM european conference on Computer Systems, pages 169–182. ACM, 2012.

[9] Cloudera impala. http://blog.cloudera.com/blog, March 2013.

[10] A. Colantonio and R. Di Pietro. Concise: Compressed 'n'composable integer set. Information Processing Letters, 110(16):644–650, 2010.

[11] J. Dean and S. Ghemawat. Mapreduce: simplified data processing on large clusters. Communications of the ACM, 51(1):107–113, 2008.

[12] G. DeCandia, D. Hastorun, M. Jampani, G. Kakulapati, A. Lakshman, A. Pilchin, S. Sivasubramanian, P. Vosshall, and W. Vogels. Dynamo: amazon's highly available key-value store. In ACM SIGOPS Operating Systems Review, volume 41, pages 205–220. ACM, 2007.

[13] C. Engle, A. Lupher, R. Xin, M. Zaharia, M. J. Franklin, S. Shenker, and I. Stoica. Shark: fast data analysis using coarse-grained distributed memory. In Proceedings of the 2012 international conference on Management of Data, pages 689–692. ACM, 2012.

[14] F. Färber, S. K. Cha, J. Primsch, C. Bornhövd, S. Sigg, and W. Lehner. Sap hana database: data management for modern business applications. ACM Sigmod Record, 40(4):45–51, 2012.

[15] B. Fink. Distributed computation on dynamo-style distributed storage: riak pipe. In Proceedings of the elEventh ACM SIGPLAN workshop on Erlang workshop, pages 43–50. ACM, 2012.

[16] B. Fitzpatrick. Distributed caching with memcached. Linux journal, (124):72–74, 2004.

[17] A. Hall, O. Bachmann, R. Büssow, S. Gănceanu, and M. Nunkesser. Processing a trillion cells per mouse click. Proceedings of the VLDB Endowment, 5(11):1436–1446, 2012.

[18] B. Hu. Stream database survey. 2011.

[19] P. Hunt, M. Konar, F. P. Junqueira, and B. Reed. Zookeeper: Wait-free coordination for internet-scale systems. In USENIX ATC, volume 10, 2010.

[20] C. S. Kim. Lrfu: A spectrum of policies that subsumes the least recently used and least frequently used policies. IEEE Transactions on Computers, 50(12), 2001.

Druid-China

[21] J. Kreps, N. Narkhede, and J. Rao. Kafka: A distributed messaging system for log processing. In Proceedings of 6th International Workshop on Networking Meets Databases (NetDB), Athens, Greece, 2011.

[22] T. Lachev. Applied Microsoft Analysis Services 2005: And Microsoft Business Intelligence Platform. Prologika Press, 2005.

[23] A. Lakshman and P. Malik. Cassandra—a decentralized structured storage system. Operating systems review, 44(2):35, 2010.

[24] Liblzf. http://freecode.com/projects/liblzf, March 2013.

[25] LinkedIn. Senseidb. http://www.senseidb.com/, July 2013.

[26] R. MacNicol and B. French. Sybase iq multiplex-designed for analytics. In Proceedings of the Thirtieth international conference on Very large data bases-Volume 30, pages 1227–1230. VLDB Endowment, 2004.

[27] N. Marz. Storm: Distributed and fault-tolerant realtime computation. http://storm-project.net/, February 2013.

[28] S. Melnik, A. Gubarev, J. J. Long, G. Romer, S. Shivakumar, M. Tolton, and T. Vassilakis. Dremel: interactive analysis of web-scale datasets. Proceedings of the VLDB Endowment, 3(1-2):330–339, 2010.

[29] D. Miner. Unified analytics platform for big data. In Proceedings of the WICSA/ECSA 2012 Companion Volume, pages 176–176. ACM, 2012.

[30] K. Oehler, J. Gruenes, C. Ilacqua, and M. Perez. IBM Cognos TM1: The Official Guide. McGraw-Hill, 2012.

[31] E. J. O'neil, P. E. O'neil, and G. Weikum. The lru-k page replacement algorithm for database disk buffering. In ACM SIGMOD Record, volume 22, pages 297–306. ACM, 1993.

[32] P. O'Neil and D. Quass. Improved query performance with variant indexes. In ACM Sigmod Record, volume 26, pages 38–49. ACM, 1997.

[33] P. O'Neil, E. Cheng, D. Gawlick, and E. O'Neil. The log-structured merge-tree (lsm-tree). Acta Informatica, 33(4):351–385, 1996.

[34] Paraccel analytic database. http://www.paraccel.com/resources/Datasheets/ParAccel-Core-Analytic-Database.pdf, March 2013.

[35] M. Schrader, D. Vlamis, M. Nader, C. Claterbos, D. Collins, M. Campbell, and F. Conrad. Oracle Essbase & Oracle OLAP. McGraw-Hill, Inc., 2009.

[36] K. Shvachko, H. Kuang, S. Radia, and R. Chansler. The hadoop distributed file system. In Mass Storage Systems and Technologies (MSST), 2010 IEEE 26th Symposium on, pages 1–10. IEEE, 2010.

[37] M. Singh and B. Leonhardi. Introduction to the ibm netezza warehouse appliance. In Proceedings of the 2011 Conference of the Center for Advanced Studies on Collaborative Research, pages 385–386. IBM Corp., 2011.

[38] M. Stonebraker, D. J. Abadi, A. Batkin, X. Chen, M. Cherniack, M. Ferreira, E. Lau, A. Lin, S. Madden, E. O'Neil, et al. C-store: a Column-oriented dbms. In Proceedings of the 31st international conference on Very large data bases, pages 553–564. VLDB Endowment, 2005.

[39] A. Tomasic and H. Garcia-Molina. Performance of inverted indices in shared-nothing distributed text document information retrieval systems. In Parallel and Distributed Information Systems, 1993., Proceedings of the Second International Conference on, pages 8–17. IEEE, 1993.

[40] E. Tschetter. Introducing Druid: Real-time analytics at a billion rows per second.

Druid-China

http://Druid.io/blog/2011/ 04/30/introducing-Druid.html, April 2011.

[41] Twitter public streams. https://dev.twitter.com/ docs/streaming-apis/streams/public, March 2013.

[42] S. J. van Schaik and O. de Moor. A memory efficient reachability data structure through bit vector compression. In Proceedings of the 2011 international conference on Management of data, pages 913–924. ACM, 2011.

[43] L. VoltDB. Voltdb technical overview. https://voltdb.com/, 2010.

[44] K. Wu, E. J. Otoo, and A. Shoshani. Optimizing Bitmap indices with efficient compression. ACM Transactions on Database Systems (TODS), 31(1):1–38, 2006.

[45] M. Zaharia, T. Das, H. Li, S. Shenker, and I. Stoica. Discretized streams: an efficient and fault-tolerant model for stream processing on large clusters. In Proceedings of the 4th USENIX conference on Hot Topics in Cloud Computing, pages 10–10. USENIX Association, 2012.



Druid-China